

7

MEMORY MANAGEMENT

7.1 Introduction

- In multiprogramming system, the task of subdividing the memory among the various processes is called memory management.
- The task of the memory management unit is the efficient utilization of memory and minimize the internal and external fragmentation.

7.2 Logical versus Physical Address

- The address generated by CPU is called the logical address.
- The address perceived by the memory unit is called physical address.

7.3 Memory Management Unit

- Hardware device that maps virtual to physical address.
- In MMU scheme, the value in the relocation register is added to every address generated by a user process at the time it is sent to memory.
- The user program deals with *logical* addresses; it never sees the *real* physical addresses.

7.3.1 Loading

It is defined as bringing the program from the secondary to the main memory.

It is classified into three types:

(i) Absolute Loading

- A given program is always loaded into the same memory location whenever loaded for execution.

(ii) Relocatable Loading

- A given program is loaded into any desired memory location whenever loaded for execution.
- The compiler must generate relative address for the program.

(iii) Dynamic Loading

- Routine is not loaded until it is called better memory-space utilization (unused routine is never loaded, postponed until execution time).
- Useful when large amounts of code are needed to handle in frequently occurring cases.
- No special support from the operating system is required. It is implemented through program design
- Address translation is performed through the hardware.

7.3.2 Linking

Linking is the process of collecting and combining various pieces of code and data into a single file that can be loaded (copied) into memory and executed. It can be performed at compile time, load time or run time.

It is classified into two types:

- **Static Linking**
Static linkers take as input a collection of relocatable object files and command-line arguments and generate as output a fully linked executable object file that can be loaded and run.
- **Dynamic Linking**
A shared library is an object module that, at run time, can be loaded at an arbitrary memory address and linked with a program in memory. This process is dynamic linking.

7.3.3 Address Binding

Association of the program instructions and data into the actual physical memory locations is called as address binding. Address binding of instructions and data to memory addresses can happen at three different stages

- **Compile time:** If memory location known a priori, **absolute code** can be generated. It must recompile code if starting location changes.
- **Load time:** It must generate **relocatable code** if memory location is not known at compile time.
- **Execution time:** Address binding is delayed until run time if the process can be moved during its execution from one memory segment to another. It needs hardware support for address mapping (e.g., base and limit registers).

7.4 Memory Management Techniques

7.4.1 Contiguous Memory Allocation

It comprises of – Fixed Partition Scheme and Variable Partition Scheme

Fixed Partition Scheme / Static Partitioning

- The memory is divided into a fixed number of partitions of equal/unequal sizes.
- Every partition is associated with limit registers.
- The degree of multiprogramming is restricted by the number of partitions.
- Partition size may not be large enough for any waiting process.
- Internal fragmentation may be present.

Variable Partition Scheme / Dynamic Partitioning

- Initially, memory is available as a single continuous free block. When a process arrives, a hole large enough to accommodate it is created in the memory.
- There is no internal fragmentation.
- It suffers from external fragmentation.
- It is associated with the overhead of compaction.

Note

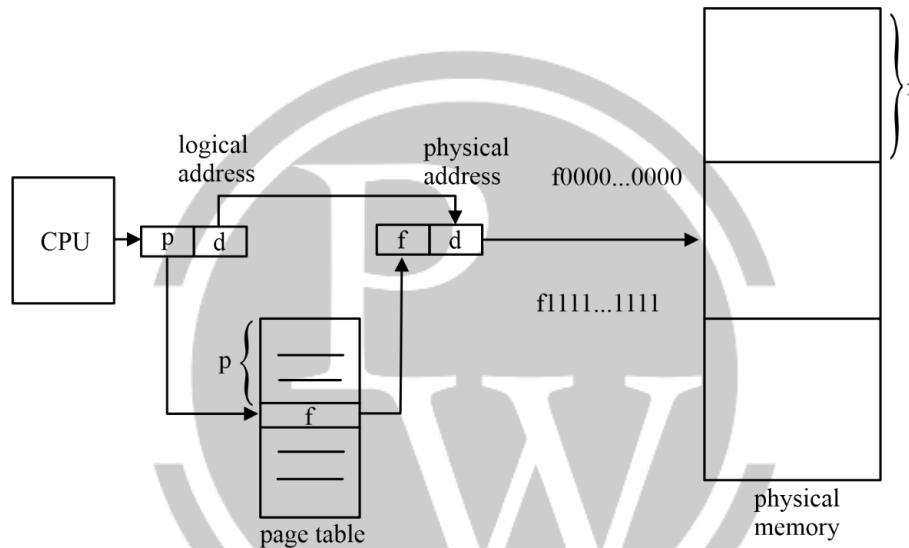
If more than one partition is sufficient to accommodate a process, then any of the following dynamic allocation methods can be adopted:

- **First Fit:** Allocate the first partition that is big enough starting from the beginning of the memory.
- **Next Fit:** It behaves similar to first fit except that it scans the memory after the 'last allocation point'.
- **Best Fit:** It scans the entire memory to find the smallest sufficient partition to accommodate the process.
- **Worst Fit:** It scans the entire memory to find the largest sufficient partition to accommodate the process.

7.4.2 Non-Contiguous Memory Allocation

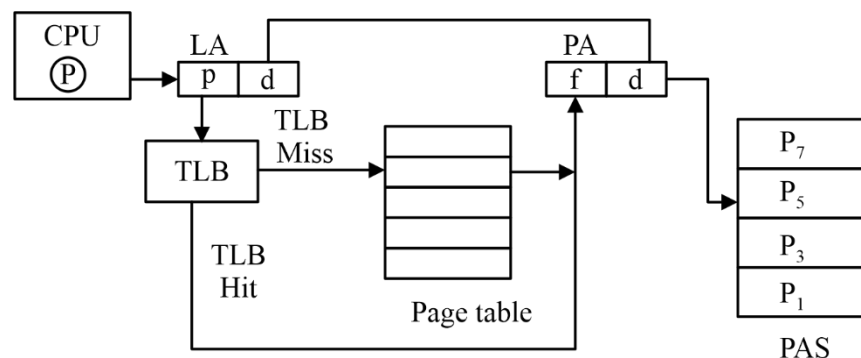
Paging

- The technique of mapping CPU generated logical address to physical address is called paging.
- Logical address space is divided into equal size pages. Physical address space is divided into equal size frames. In paging, the frame size is equal to the page size.
- When a process is created, paging will be applied on the process. A page table will be maintained in the main memory for a process. The base address of the page table will be stored in the process control block.
- The number of entries in the page table is equal to the number of the pages in the logical address space. Each page table entry contains the frame number. Hence, the page table is also known as the **address translation table**.
- There is no external fragmentation in paging. Internal fragmentation may exist in the last page and is formulated as $\left\lfloor \frac{p}{2} \right\rfloor$ where p is the page size.



Paging with TLB

- **Translation Look Aside Buffer** is a hardware device implemented using associative registers.
- **TLB** is added to improve the performance of paging. The TLB access time will be very less compared to the main memory access time.
- **TLB** contains frequently referred page numbers and their corresponding frame numbers.



- TLB access time = c
- TLB hit ratio = x

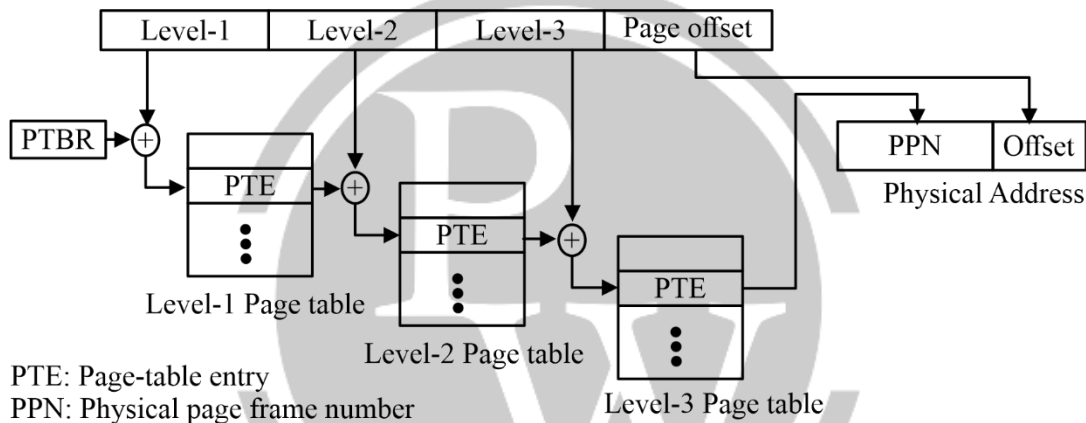
Then the formula for effective memory access time:

$$E.M.A.T = x(c + m) + (1 - x) \cdot 1 \cdot (c + 2m)$$

1 TLB access
1 MR for actual page
1 MR for PT
1 MR for actual page

Multilevel Paging

- To avoid the overhead of maintaining large size page tables, multilevel paging will be applied.
- In multilevel paging, pages of the page table are brought into the main memory.
- The page tables of all the levels of multi-level paging are kept in the memory.
- The entries of the level-1 page table are pointers to the level-2 page table.
- The entries of the level-2 page table are pointers to the level-3 page table.
- The entries of the last level page table will have frame numbers of the actual page.
- All levels page table entry must contain frame number.
- Address translation is done as:

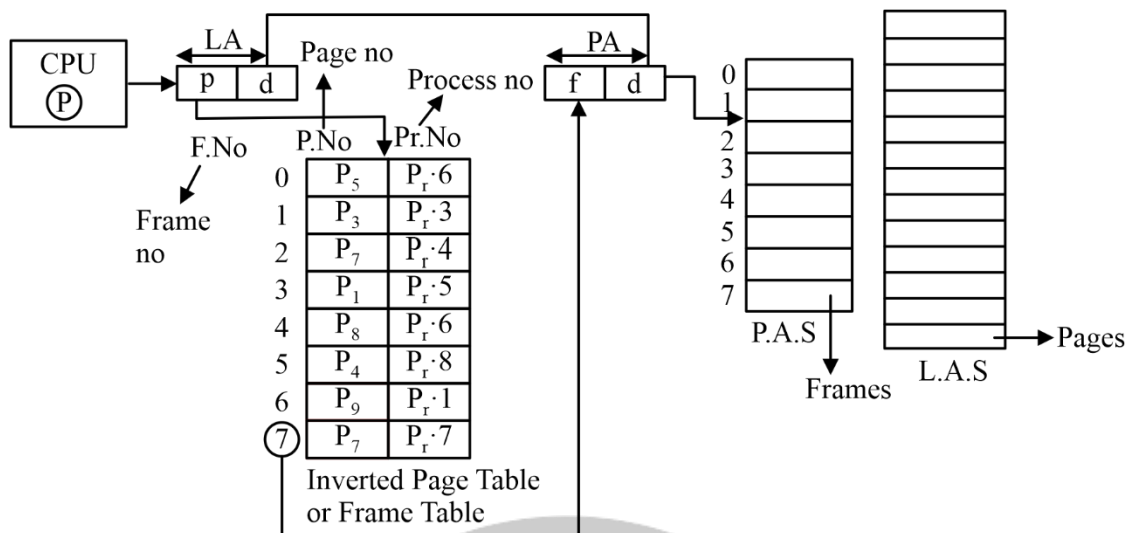


- Performance of multi-level paging with TLB:
Assume, TLB hit ratio = p , TLB access time = c , Main Memory Access time = m
If the system uses n -level paging, then effective memory access time is given as-
$$EMAT = p(c + m) + (1 - p)(c + (n + 1)m)$$

Inverted Paging

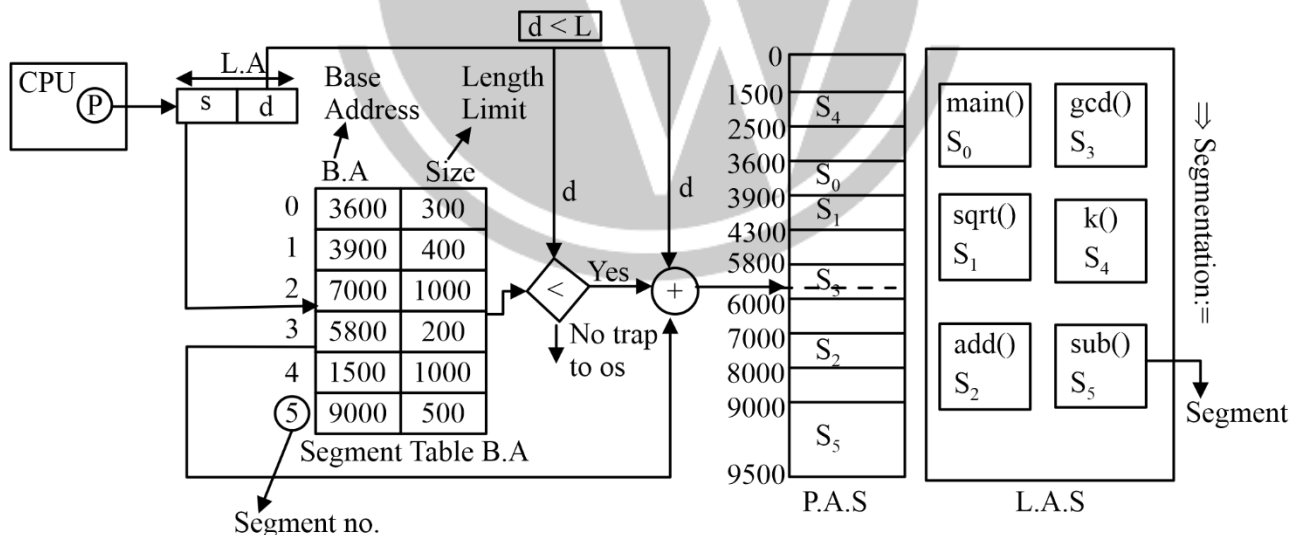
- Inverted paging maps physical frames to virtual pages.
- Instead of maintaining multiple page tables for different processes, an inverted page table can be maintained.
- The number of entries in the inverted page table is equal to the number of the frames in the physical address space.
- Each inverted page table entry contains a page number and a process identifier. It gives an idea about 'which page of which process is allocated in which frame'.
- The disadvantage of inverted paging is the increased lookup time and hard to implement.

- Address translation is done as:



Segmentation

- Paging does not follow user's view of memory allocation. Instead of dividing the process into equal sized pages, it can be divided into variable length segments.
- A segment table is maintained for each process. The number of entries in the segment table is equal to the number of segments of a process.
- Each segment table entry contains the base (starting address) and limit (length) of the segment.
- Segmentation does not suffer from internal or external fragmentation.
- Address translation is done as:



Segmented Paging:

- To avoid the overhead of bringing large sized segments of a process into the main memory, paging will be applied on the segments.
- Pages of the segment will be loaded into the main memory.
- A page table will be maintained for each segment of the process.
- The number of entries in the page table is equal to the number of the pages of the segment in the logical address space.
- Lookup time increases.

- It suffers from internal fragmentation only.
- Address translation is done as:

